

INSERTION SORT

Date : 10 / 07 / 2025

Topic : Insertion Sort

1. INTRODUCTION

Insertion Sort is a simple sorting algorithm that builds the final sorted array one item at a time. It picks each element and inserts it into its correct position in the sorted part of the array. This process is repeated until the entire array is sorted.

Key Characteristics

- In-place
- Stable
- Simple to implement
- Efficient for small or nearly sorted arrays

2. IDEA / INTUITION

- Assume the first element is already sorted.
- Take the next element (key) and compare it with elements in the sorted part.
- Shift elements that are greater than the key one position ahead.
- Insert the key at its correct position.
- Repeat for all elements.

Example : arr = [5, 2, 4, 6, 1, 3]

3. ALGORITHM

1. Start from the second element (index 1).
2. Take the current element as key.
3. Compare the key with elements in the sorted part (left side).
4. Shift all greater elements one position ahead.
5. Insert the key at its correct position.
6. Repeat steps 2 to 5 for all elements.
7. Stop.

4. PSEUDOCODE

```
InsertionSort(arr)
n ← length(arr)
for i ← 1 to n-1      // Start from 2nd element
    key ← arr[i]
    j ← i - 1          // Index of last element in
                       // sorted part
    while j ≥ 0 and arr[j] > key
        arr[j + 1] ← arr[j]  // Shift greater
                             // element to right
        j ← j - 1
    arr[j + 1] ← key        // Insert key at
                           // correct position
```

5. EXAMPLE DRY RUN

Let the array be : [5, 2, 4, 6, 1, 3]

Pass 0 (i = 1) : key = 2

[5 | 2 | 4 | 6 | 3] 2 < 5 ⇒ shift 5
↓
[2 | 5 | 4 | 6 | 3] Insert 2

Pass 1 (i = 2) : key = 4

[2 | 5 | 4 | 6 | 3] 4 < 5 ⇒ shift 5
↓
[2 | 4 | 5 | 6 | 3] Insert 4

Pass 2 (i = 3) : key = 6

[2 | 4 | 5 | 6 | 3] 6 > 5 ⇒ no shift
↓
[2 | 4 | 5 | 6 | 3] Already in place

(Further passes on next page)

6. TIME & SPACE COMPLEXITY

Case	Time Complexity
Best Case (Already Sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case (Reverse Sorted)	$O(n^2)$
Space Complexity	$O(1)$

($O(1)$ because it sorts in-place, no extra space used)

7. ADVANTAGES

- ✓ Simple and easy to implement.
- ✓ In-place sorting (uses $O(1)$ extra space).
- ✓ Stable algorithm.
- ✓ Efficient for small or nearly sorted arrays.
- ✓ Online algorithm (can sort as data arrives).

8. IMPORTANT NOTE



Insertion Sort is efficient when the array is already sorted or has only a few elements out of place. It performs poorly on large, reverse sorted arrays.

Insertion Sort builds the sorted array one element at a time!



INSERTION SORT - PAGE 2

Date : 10 / 07 / 2025

Topic : Insertion Sort

1. EXAMPLE DRY RUN (CONTINUED)

Let the array be : 5, 2, 4, 6, 1, 3

Pass 3 ($i = 4$): key = 1

2 4 5 6 1 3 $1 < 6 \Rightarrow$ shift 6

2 4 5 1 6 3 $1 < 5 \Rightarrow$ shift 5

2 4 1 5 6 3 $1 < 4 \Rightarrow$ shift 4

2 1 4 5 6 3 $1 < 2 \Rightarrow$ shift 2

Pass 3 result : [1, 2, 4, 5, 6, 3] Insert 1

Pass 4 ($i = 5$): key = 3

1 2 4 5 6 3 $3 < 6 \Rightarrow$ shift 6

1 2 4 5 3 6 $3 < 5 \Rightarrow$ shift 5

1 2 4 3 5 6 $3 < 4 \Rightarrow$ shift 4

1 2 3 4 5 6 $3 > 2 \Rightarrow$ stop

Pass 4 result : [1, 2, 3, 4, 5, 6] Sorted Array

2. ACTUAL CODE (C)

```
#include <stdio.h>
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) { // Start from 2nd element
        int key = arr[i]; // Element to insert
        int j = i - 1; // Index of last element in sorted part
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // Shift to right
            j = j - 1;
        }
        arr[j + 1] = key; // Insert key
    }
}

int main() {
    int arr[] = {5, 2, 4, 6, 1, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    insertionSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

How it works?

- Pick next element (key)
- Shift greater elements to right
- Insert key in its correct position
- Repeat until array is sorted

3. VISUAL SUMMARY

Pass	i (index)	Key	Array after pass	Sorted Part
Initial	-	-	5 2 4 6 1 3	-
0	1	2	2 5 4 6 1 3	[2, 5]
1	2	4	2 4 5 6 1 3	[2, 4, 5]
2	3	6	2 4 5 6 1 3	[2, 4, 5, 6]
3	4	1	1 2 4 5 6 3	[1, 2, 4, 5, 6]
4	5	3	1 2 3 4 5 6	[1, 2, 3, 4, 5, 6]

4. COMPLEXITY ANALYSIS

- Best Case (Already Sorted) : $O(n)$
- Average Case : $O(n^2)$
- Worst Case (Reverse Sorted) : $O(n^2)$
- Space Complexity : $O(1)$

Why $O(n)$ in Best Case?

If the array is already sorted, the while loop never executes. We only compare elements, so total operations are proportional to n .



5. PROPERTIES

- In-place algorithm (uses $O(1)$ extra space).
- Stable sorting algorithm.
- Adaptive - runs faster on nearly sorted arrays.
- Online algorithm - can sort as data arrives.

6. ADVANTAGES

- ✓ Easy to understand and implement.
- ✓ Efficient for small or nearly sorted arrays.
- ✓ Requires only one pass of array in best case.
- ✓ In-place and stable.

7. DISADVANTAGES

- ✗ Inefficient for large datasets.
- ✗ More comparisons and shifts in average and worst case.
- ✗ Not suitable for reverse sorted arrays.

8. IMPORTANT POINTS (EXAMS/INTERVIEWS)

- * Insertion Sort builds the sorted array one element at a time.
- * It maintains a sorted left part and inserts new element in it.
- * Minimum number of swaps (actually uses shifts instead of swaps).
- * Particularly useful when input size is small or array is almost sorted.
- * Time complexity is $O(n)$ for best case and $O(n^2)$ for average/worst.

9. COMMON MISTAKES

- Not initializing $j = i - 1$.
- Using ' $>$ ' instead of ' $>=$ ' in while condition.
- Forgetting to place key at $arr[j + 1]$.
- Off-by-one errors in loop boundaries.



Insertion Sort is simple, adaptive and efficient for small or nearly sorted data! ♥